

	Matière : Atelier C++ TP 3	Année Scolaire 2025-2026 Enseignante : Amani FALLAH
---	--	---

Exercice 1 : allocation statique / allocation dynamique

On veut créer un tableau d'entier, le remplir avec des valeurs entières puis on affichera le nombre de valeurs entières positives. Pensez à diviser le problème en fonctions.

Observez votre code et optimisez le si c'est possible en termes de temps d'exécution.

◆ Version 1 — Allocation statique

```
#include <iostream>
using namespace std;

// Fonction pour compter les valeurs positives ◇ Version 1 - Allocation statique
int compterPositifs(const int tab[], int n) {
    int count = 0;
    for (int i = 0; i < n; ++i)
        if (tab[i] > 0)
            count++;
    return count;
}

int main() {
    const int n = 5;
    int tab[n]; // allocation statique

    cout << "Entrez " << n << " entiers : ";
    for (int i = 0; i < n; ++i)
        cin >> tab[i];

    cout << "Nombre de valeurs positives : " << compterPositifs(tab, n) << endl;
    return 0;
}
```

◆ Version 2 — Allocation dynamique

```
// Fonction pour compter les valeurs positives ◇ Version 2 - Allocation
dynamique

#include <iostream>
using namespace std;

int compterPositifs(const int* tab, int n) {
    int count = 0;
    for (int i = 0; i < n; ++i)
        if (tab[i] > 0)
            count++;
    return count;
}
```

```

int main() {
    int n;
    cout << "Donnez la taille du tableau : ";
    cin >> n;

    int* tab = new int[n]; // allocation dynamique

    cout << "Entrez " << n << " entiers : ";
    for (int i = 0; i < n; ++i)
        cin >> tab[i];

    cout << "Nombre de valeurs positives : " << compterPositifs(tab, n) << endl;

    delete[] tab; // libération mémoire
    return 0;
}

```

➔ **Optimisation** : une seule boucle pour saisir et compter peut suffire (gain de temps et de mémoire).

Exercice 2 :

Écrire une fonction remplir qui permet de réserver et de remplir un tableau de N entiers.
 Écrire une fonction recherche qui recherche une valeur entière V dans un tableau. La fonction doit rechercher si V se trouve dans le tableau et afficher « V se trouve dans le tableau à la position i » ou « V ne se trouve pas dans le tableau ». Écrire un programme principale main qui exploite les fonctions précédemment définies. (Donnez une version avec les tableaux bruts, puis une version avec l'objet std::array)

Reprendre le même problème et le résoudre à l'aide du conteneur vector.

Version 1 :

```

#include <iostream>
using namespace std;

int* remplir(int n) {
    int* tab = new int[n];
    cout << "Entrez " << n << " entiers : ";
    for (int i = 0; i < n; ++i)
        cin >> tab[i];
    return tab;
}

void recherche(int* tab, int n, int v) {
    for (int i = 0; i < n; ++i) {
        if (tab[i] == v) {
            cout << v << " se trouve dans le tableau à la position " << i <<
endl;
            return;
        }
    }
    cout << v << " ne se trouve pas dans le tableau." << endl;
}

int main() {
    int n;

```

```

    cout << "Taille du tableau : ";
    cin >> n;

    int* tab = remplir(n);

    int v;
    cout << "Entrez la valeur à rechercher : ";
    cin >> v;

    recherche(tab, n, v);

    delete[] tab;
    return 0;
}

```

Version 2 (std::array):

```

#include <iostream>
#include <array>
using namespace std;

template<size_t N>
void remplir(array<int, N>& tab) {
    cout << "Entrez " << N << " entiers : ";
    for (auto& val : tab)
        cin >> val;
}

template<size_t N>
void recherche(const array<int, N>& tab, int v) {
    for (size_t i = 0; i < N; ++i) {
        if (tab[i] == v) {
            cout << v << " se trouve à la position " << i << endl;
            return;
        }
    }
    cout << v << " ne se trouve pas dans le tableau." << endl;
}

int main() {
    array<int, 5> tab;
    remplir(tab);

    int v;
    cout << "Valeur à rechercher : ";
    cin >> v;

    recherche(tab, v);
    return 0;
}

```

Version 3 (vector):

```

#include <iostream>
#include <vector>
using namespace std;

vector<int> remplir(int n) {
    vector<int> tab(n);
    cout << "Entrez " << n << " entiers : ";
    for (int i = 0; i < n; ++i)

```

```

        cin >> tab[i];
    return tab;
}

void recherche(const vector<int>& tab, int v) {
    for (size_t i = 0; i < tab.size(); ++i) {
        if (tab[i] == v) {
            cout << v << " se trouve à la position " << i << endl;
            return;
        }
    }
    cout << v << " ne se trouve pas dans le tableau." << endl;
}

int main() {
    int n;
    cout << "Taille du tableau : ";
    cin >> n;

    vector<int> tab = remplir(n);

    int v;
    cout << "Valeur à rechercher : ";
    cin >> v;

    recherche(tab, v);
    return 0;
}

```

Exercice 3 :

Ecrire un programme qui calcule le produit scalaire de deux vecteurs d'entiers U et V (de même dimension).

Exemple :

$U=\{1,2,3\}$ et $V=\{4,5,6\}$ donc $U*V=1*4+2*5+3*6=32$

```

#include <iostream>
using namespace std;

int produitScalaire(const int* U, const int* V, int n) {
    int produit = 0;
    for (int i = 0; i < n; ++i)
        produit += U[i] * V[i];
    return produit;
}

int main() {
    int n;
    cout << "Taille des vecteurs : ";
    cin >> n;

    int* U = new int[n];
    int* V = new int[n];

    cout << "Entrez les éléments de U : ";
    for (int i = 0; i < n; ++i) cin >> U[i];
}

```

```

    cout << "Entrez les éléments de V : ";
    for (int i = 0; i < n; ++i) cin >> V[i];

    cout << "Produit scalaire = " << produitScalaire(U, V, n) << endl;

    delete[] U;
    delete[] V;
    return 0;
}

```

Exercice 4 : Fonctions retournant plusieurs valeurs (std::pair, std::tuple)

Ecrire une fonction mini_maxi qui retourne la plus grande et la plus petite valeur dans un tableau d'entiers A.

Dans la fonction main, afficher les valeurs retournées par la fonction mini_maxi. Pour remplir le tableau utiliser les fonctions définies dans les exercices précédents.

Version avec std::pair

```

#include <iostream>
#include <vector>
#include <utility> // pour std::pair
using namespace std;

// Fonction mini_maxi qui retourne (min, max)
pair<int, int> mini_maxi(const vector<int>& A) {
    int minVal = A[0], maxVal = A[0];
    for (int val : A) {
        if (val < minVal) minVal = val;
        if (val > maxVal) maxVal = val;
    }
    return make_pair(minVal, maxVal);
}

int main() {
    int n;
    cout << "Taille du tableau : ";
    cin >> n;

    vector<int> A(n);
    cout << "Entrez " << n << " entiers : ";
    for (int& x : A) cin >> x;

    auto result = mini_maxi(A);
    cout << "Valeur minimale : " << result.first << endl;
    cout << "Valeur maximale : " << result.second << endl;

    return 0;
}

```

Version avec std::tuple

```

#include <iostream>
#include <vector>
#include <tuple> // pour std::tuple
using namespace std;

```

```

tuple<int, int> mini_maxi(const vector<int>& A) {
    int minVal = A[0], maxVal = A[0];
    for (int val : A) {
        if (val < minVal) minVal = val;
        if (val > maxVal) maxVal = val;
    }
    return make_tuple(minVal, maxVal);
}

int main() {
    int n;
    cout << "Taille du tableau : ";
    cin >> n;

    vector<int> A(n);
    cout << "Entrez " << n << " entiers : ";
    for (int& x : A) cin >> x;

    auto [minVal, maxVal] = mini_maxi(A); // décomposition structurée
    cout << "Min : " << minVal << ", Max : " << maxVal << endl;

    return 0;
}

```

Exercice 5 : tableau dynamique / map

Soit la structure ville définie par un nom et une position (longitude et latitude). On veut gérer l'ensemble des villes de la Tunisie et créer un programme qui recherche la ville par son nom et affiche sa position.

Donnez une version avec les vector puis une autre version avec unordered_map. Comparer les performances de chaque méthode.

Version 1 (vector) :

```

#include <iostream>
#include <vector>
#include <string>
using namespace std;

struct Ville {
    string nom;
    double latitude;
    double longitude;
};

void ajouterVille(vector<Ville>& villes, const string& nom, double lat, double lon) {
    villes.push_back({ nom, lat, lon });
}

void rechercherVille(const vector<Ville>& villes, const string& nom) {
    for (const auto& v : villes) {
        if (v.nom == nom) {
            cout << nom << " → (" << v.latitude << ", " << v.longitude << ")" << endl;
            return;
        }
    }
    cout << nom << " non trouvée !" << endl;
}

```

```

}

int main() {
    vector<Ville> villes;

    ajouterVille(villes, "Tunis", 36.8065, 10.1815);
    ajouterVille(villes, "Sfax", 34.7406, 10.7603);
    ajouterVille(villes, "Sousse", 35.8256, 10.6411);

    string nom;
    cout << "Entrez le nom de la ville à rechercher : ";
    cin >> nom;

    rechercherVille(villes, nom);
    return 0;
}

```

Version 2 (unordered map) :

```

#include <iostream>
#include <unordered_map>
#include <string>
using namespace std;

struct Position {
    double latitude;
    double longitude;
};

int main() {
    unordered_map<string, Position> villes;

    villes["Tunis"] = { 36.8065, 10.1815 };
    villes["Sfax"] = { 34.7406, 10.7603 };
    villes["Sousse"] = { 35.8256, 10.6411 };

    string nom;
    cout << "Entrez le nom de la ville : ";
    cin >> nom;

    auto it = villes.find(nom);
    if (it != villes.end()) {
        cout << nom << " → (" << it->second.latitude << ", " << it-
>second.longitude << ")" << endl;
    }
    else {
        cout << nom << " non trouvée !" << endl;
    }

    return 0;
}

```

Comparaison des performances

Structure	Type	Temps moyen d'accès	Complexité
vector	Tableau dynamique	linéaire	doit parcourir tout le tableau
unordered_map	Table de hachage	constante	très rapide pour grandes données

➔ Pour une **grande quantité de villes**, unordered_map est **nettement plus performant**.